

Using the dataframe `data` print the first 5 rows

failings	links	static_fire_date_utc	static_fire_date_unix	tbd	net	window	rocket	success	details	crew	ships	capsules	payloads	launchpad	auto_update	failures	flight_number	name
0	[patch: {small: https://images2.imgbox.com/3c/0e/78bC5N3_o.png, large: https://images2.imgbox.com/40/43/d9p5tXayF_o.png}, reddit: {campaign: None, launch: None, media: None, recovery: None}, flickr: {small: [], original: []}, presskit: None, webcast: https://www.youtube.com/watch?v=QduuY_VB8, youtube_id: '3a_dmo_Y88', article: https://www.space.com/2195-spaces-magnum-falcon-1-rocket-tot-launch.html', wikipedia: https://en.wikipedia.org/wiki/DemoSat}	2006-03-17T00:00:00.000Z	1.422545e+09	False	False	0.0	SeaOrb95da5a9959577091eb	False	Engine failure at 33 seconds and loss of vehicle				[SeaOrb4db6bc3bc0c00e6b1e4], SeaOrb4502f93009956a566866	True		[{time: 33, altitude: None, reason: 'main engine failure'}]	1	FalconSat
	[patch: {small: https://images2.imgbox.com/46/a3/d6uLk2_o.png, large: https://images2.imgbox.com/bef/7/Neqz7MYL_o.png}, reddit: {campaign: None, launch: None, media: None, recovery: None}, flickr: {small: [], original: []}, presskit: None, webcast: https://www.youtube.com/watch?v=Lk4Q2uPw-Nc, youtube_id: 'Lk4Q2uPw-Nc', article: https://www.space.com/3350-spacex-falcon-1-rocket-fails-reach-orbit.html', wikipedia: https://en.wikipedia.org/wiki/DemoSat}	None	NaN	False	False	0.0	SeaOrb95da5a9959577091eb	False	Successful first stage burn and transition to second stage, maximum altitude 289 km				[SeaOrb4db6bc3bc0c00e6b1e4], SeaOrb4502f93009956a566866	True		[{time: 301, altitude: 289, reason: 'harmonic oscillation leading to premature engine shutdown'}]	2	DemoSat
	[patch: {small: https://images2.imgbox.com/3d/86/cn0pan8_o.png, large: https://images2.imgbox.com/4b/4d/d8dUkL42_o.png}, reddit: {campaign: None, launch: None, media: None, recovery: None}, flickr: {small: [], original: []}, presskit: None, webcast: https://www.youtube.com/watch?v=V0w9r3U8860, youtube_id: 'V0w9r3U8860', article: https://www.space.com/news/2013/02/11/falcon-1-fights-3-mission-summary', wikipedia: https://en.wikipedia.org/wiki/TraillizerSat}	None	NaN	False	False	0.0	SeaOrb95da5a9959577091eb	False	Residual stage 1 thrust led to collision between stage 1 and stage 2				[SeaOrb4db6bc3bc0c00e6b1e4], SeaOrb4502f93009956a566866	True		[{time: 140, altitude: 35, reason: 'residual stage 1 thrust led to collision between stage 1 and stage 2'}]	3	TraillizerSat

[illegible]

We will now use the API again to get information about the launches using the IDs given for each launch. Specifically we will be using columns `rocket`, `payloads`, `launchpad`, and `cores`.

- From the `rocket` we would like to learn the booster name
- From the `payload` we would like to learn the mass of the payload and the orbit that it is going to
- From the `launchpad` we would like to know the name of the launch site being used, the longitude, and the latitude.
- From `cores` we would like to learn the outcome of the landing, the type of the landing, number of flights with that core, whether griffins were used, whether the core is reused, whether legs were used, the landing pad used, the block of the core which is a number used to separate version of cores, the number of times this specific core has been reused, and the serial of the core.

```

# Define variables
BoosterVersion = []
PayloadMass = []
Orbit = []
LaunchSite = []
Outcome = []
Flights = []
Gridfins = []
Reused = []
Legs = []
LandingPad = []
Block = []
ReusedCount = []
Serial = []
Longitude = []
Latitude = []

# Create empty lists to store the data
BoosterVersion = []
PayloadMass = []
Orbit = []
LaunchSite = []
Outcome = []
Flights = []
Gridfins = []
Reused = []
Legs = []
LandingPad = []
Block = []
ReusedCount = []
Serial = []
Longitude = []
Latitude = []

# Call the helper functions to populate the lists
getBoosterVersion(df)
getPayloadMass(df)
getLaunchSite(df)
getOutcome(df)

# Create a new dataframe with all the collected data
launch_dict = {'FlightNumber': list(df['flight_number']),
               'Date': list(df['date']),
               'BoosterVersion': BoosterVersion,
               'PayloadMass': PayloadMass,
               'Orbit': Orbit,
               'LaunchSite': LaunchSite,
               'Outcome': Outcome,
               'Flights': Flights,
               'Gridfins': Gridfins,
               'Reused': Reused,
               'Legs': Legs,
               'LandingPad': LandingPad,
               'Block': Block,
               'ReusedCount': ReusedCount,
               'Serial': Serial,
               'Longitude': Longitude,
               'Latitude': Latitude}

# Create a dataframe from the dictionary
launch_df = pd.DataFrame(launch_dict)

# Display the first few rows of the new dataframe
launch_df

```

[]: `BoosterVersion`

Now, let's apply `getBoosterVersion` function method to get the booster version

we can apply the rest of the functions here:

```
[ ]: # Call getPayloadData
getPayloadData(data)
```

```
[ ]: # Call getCoreData
getCoreData(data)
```

```
[2]: # Create a DataFrame from the dictionary
launch_df = pd.DataFrame(launch_dict)

# Display the first few rows to verify the data
launch_df.head()
```

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_75/2853379576.py in <module>
      1 # Create a DataFrame from the dictionary
----> 2 launch_df = pd.DataFrame(launch_dict)
      3
      4 # Display the first few rows to verify the data
      5 launch_df.head()

NameError: name 'pd' is not defined
```

Then, we need to create a Pandas data frame from the dictionary launch_dict.

```
[3]: # Create a DataFrame from launch_dict
launch_df = pd.DataFrame(launch_dict)

# Display the first 5 rows to verify the data was imported correctly
launch_df.head()
```

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_75/4278134684.py in <module>
      1 # Create a DataFrame from launch_dict
----> 2 launch_df = pd.DataFrame(launch_dict)
      3
      4 # Display the first 5 rows to verify the data was imported correctly
      5 launch_df.head()

NameError: name 'pd' is not defined
```

Show the summary of the dataframe

```
[4]: # Show the head of the dataframe
```

Task 2: Filter the dataframe to only include Falcon 9 launches

Finally we will remove the Falcon 1 launches keeping only the Falcon 9 launches. Filter the data dataframe using the `BoosterVersion` column to only keep the Falcon 9 launches. Save the filtered data to a new dataframe called `data_falcon9`.

```
[28]: # Filter the DataFrame to include only Falcon 9 launches
data_falcon9 = launch_df[launch_df['BoosterVersion'] != 'Falcon 1']

# Display the first few rows of the filtered DataFrame
data_falcon9.head()
```

```
# Verify the filtering worked by checking for any 'Falcon 1' entries
print("Number of Falcon 1 launches remaining:",
      (data_falcon9['BoosterVersion'] == 'Falcon 1').sum())

# Check the shape of the filtered DataFrame
print("Filtered DataFrame shape:", data_falcon9.shape)
```

Number of Falcon 1 launches remaining: 0
Filtered DataFrame shape: (90, 17)

Now that we have removed some values we should reset the `FlightNumber` column

```
[27]: data_falcon9.loc[:, 'FlightNumber'] = list(range(1, data_falcon9.shape[0]+1))
data_falcon9
```

```
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages/pandas/core/indexing.py:1775: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
self._setitem_single_column(locs[0], value, pi)
```

```
[27]:
FlightNumber    Date  BoosterVersion  PayloadMass  Orbit  LaunchSite  Outcome  Flights  Gridfins  Reused  Legs  LandingPad  Block  ReusedCount  Serial  Longitude  Latitude
4      1  2010-06-04      Falcon 9      NaN      LEO      CCSPS  SLC 40      None  None      1      False  False  False      None  1.0      0      B0003      -80.577366  28.561857
5      2  2012-05-22      Falcon 9      525.0  LEO      CCSPS  SLC 40      None  None      1      False  False  False      None  1.0      0      B0005      -80.577366  28.561857
6      3  2013-03-01      Falcon 9      677.0  ISS      CCSPS  SLC 40      None  None      1      False  False  False      None  1.0      0      B0007      -80.577366  28.561857
7      4  2013-09-29      Falcon 9      500.0  PO      VABF  SLC 4E      False  Ocean      1      False  False  False      None  1.0      0      B1003      -120.610829  34.632093
8      5  2013-12-03      Falcon 9      3170.0  GTO      CCSPS  SLC 40      None  None      1      False  False  False      None  1.0      0      B1004      -80.577366  28.561857
--      --      --      --      --      --      --      --      --      --      --      --      --      --      --      --      --
89     86  2020-09-03      Falcon 9      15800.0  VLEO      KSC  LC 39A      True  ASDS      2      True  True  True  569e302383ecdbdb234a7ca  5.0      12      B1060      -80.603956  28.608058
90     87  2020-10-06      Falcon 9      15800.0  VLEO      KSC  LC 39A      True  ASDS      3      True  True  True  569e302383ecdbdb234a7ca  5.0      13      B1058      -80.603956  28.608058
91     88  2020-10-18      Falcon 9      15800.0  VLEO      KSC  LC 39A      True  ASDS      6      True  True  True  569e302383ecdbdb234a7ca  5.0      12      B1051      -80.603956  28.608058
92     89  2020-10-24      Falcon 9      15800.0  VLEO      CCSPS  SLC 40      True  ASDS      3      True  True  True  569e303383ecdb9e334a7cc  5.0      12      B1060      -80.577366  28.561857
93     90  2021-11-05      Falcon 9      3681.0  MEO      CCSPS  SLC 40      True  ASDS      1      True  False  True  569e302383ecdbdb234a7ca  5.0      8      B1062      -80.577366  28.561857
```

90 rows x 17 columns

Data Wrangling

We can see below that some of the rows are missing values in our dataset.

```
[28]: data_falcon9.isnull().sum()
```

```
[28]:
FlightNumber    0
Date            0
BoosterVersion  0
PayloadMass     5
Orbit           0
LaunchSite      0
Outcome         0
Flights         0
Gridfins        0
Reused          0
Legs            0
LandingPad      26
Block           0
ReusedCount     0
Serial          0
Longitude       0
Latitude        0
dtype: int64
```

Before we can continue we must deal with these missing values. The `LandingPad` column will retain `None` values to represent when landing pads were not used.

Task 3: Dealing with Missing Values

Calculate below the mean for the `PayloadMass` using the `.mean()`. Then use the mean and the `.replace()` function to replace `np.nan` values in the data with the mean you calculated.

```
[29]: # Calculate the mean value of PayloadMass column
payload_mass_mean = data_falcon9['PayloadMass'].mean()
print("Mean PayloadMass:", payload_mass_mean)

# Replace the np.nan values with its mean value
data_falcon9['PayloadMass'].replace(np.nan, payload_mass_mean, inplace=True)

# Verify there are no more NaN values in PayloadMass column
print("Number of NaN values remaining:", data_falcon9['PayloadMass'].isna().sum())
```

Mean PayloadMass: 6123.547647858824

Number of NaN values remaining: 0

```
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages/pandas/core/generic.py:1663: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
return self._update_inplace(result)
```

```
[30]: # Create an explicit copy when filtering
data_falcon9 = launch_df[launch_df['BoosterVersion'] != 'Falcon 1'].copy()

# Calculate the mean value of PayloadMass column
payload_mass_mean = data_falcon9['PayloadMass'].mean()
print("Mean PayloadMass:", payload_mass_mean)

# Replace the np.nan values with its mean value
data_falcon9['PayloadMass'].replace(np.nan, payload_mass_mean, inplace=True)

# Verify there are no more NaN values in PayloadMass column
print("Number of NaN values remaining:", data_falcon9['PayloadMass'].isna().sum())
```

Mean PayloadMass: 6123.547647858824

Number of NaN values remaining: 0

You should see the number of missing values of the `PayloadMass` change to zero.

Now we should have no missing values in our dataset except for in `LandingPad`.

We can now export it to a CSV for the next section, but to make the answers consistent, in the next lab we will provide data in a pre-selected date range.

```
[31]: # Export the DataFrame to a CSV file
data_falcon9.to_csv('falcon9_dataset.csv', index=False)
print("Data successfully exported to 'falcon9_dataset.csv'")

Data successfully exported to 'falcon9_dataset.csv'

data_falcon9.to_csv('dataset_part_1.csv', index=False)
```

Authors

Joseph Santarangelo has a PhD in Electrical Engineering, his research focused on using machine learning, signal processing, and computer vision to determine how videos impact human cognition. Joseph has been working for IBM since he completed his PhD.

